

Informe Exhaustivo sobre Mejores Prácticas para la Estructura de Repositorios GitHub, Organización de Documentación y Gestión de Archivos de Código

Introducción

La gestión eficaz de repositorios de código es fundamental para el éxito de cualquier proyecto de software, especialmente en entornos colaborativos y con stacks tecnológicos diversos. Un repositorio bien estructurado no solo facilita la incorporación de nuevos miembros al equipo, sino que también mejora la mantenibilidad, la escalabilidad y la comprensión general del proyecto. Este informe detalla las mejores prácticas para la estructura de repositorios GitHub, la organización de la documentación y la gestión de archivos de código, abordando las necesidades de un stack tecnológico amplio que incluye Python, Java/Kotlin, TypeScript, Rust, React, Spring Boot, FastAPI y Docker.

1. Mejores Prácticas para la Estructura del Repositorio

Una estructura de repositorio clara y consistente es la base para un proyecto bien organizado. Ayuda a los desarrolladores a localizar rápidamente el código, la configuración y la documentación relevantes, y fomenta un flujo de trabajo eficiente ¹.

Estructuras de Carpetas Estándar para Diferentes Tipos de Proyectos

La estructura ideal de un repositorio puede variar significativamente según el tipo de proyecto. A continuación, se presentan algunas estructuras comunes:

- **Fullstack:** Para aplicaciones que combinan frontend y backend, una estructura común separa estas dos capas, a menudo con una carpeta `shared` para el código común.

Plain Text

```
.
├── backend/                # Código del servidor (e.g., Spring Boot, FastAPI)
│   ├── src/
│   ├── pom.xml / requirements.txt
│   └── ...
├── frontend/              # Código del cliente (e.g., React, TypeScript)
│   ├── public/
│   ├── src/
│   └── package.json
```

```

|   └─ ...
|   └─ shared/           # Código, tipos o configuraciones compartidas
|       └─ utils/
|       └─ types/
|   └─ docs/             # Documentación del proyecto
|   └─ .github/          # Configuraciones de GitHub (CI/CD, plantillas)
|   └─ .gitignore
|   └─ README.md
|   └─ ...

```

- **ML/AI:** Los proyectos de Machine Learning e Inteligencia Artificial requieren una organización específica para datos, modelos y experimentos.

Plain Text

```

.
├─ data/                 # Datos brutos, procesados y externos
│   ├── 01_raw/          # Datos originales, inmutables
│   ├── 02_intermediate/ # Datos transformados, intermedios
│   ├── 03_primary/      # Datos limpios y finales para modelado
│   └─ 04_models/        # Datos de entrada/salida de modelos
├─ notebooks/           # Jupyter notebooks para experimentación y análisis
├─ src/                 # Código fuente de modelos, pipelines, utilidades
│   ├── models/
│   ├── features/
│   ├── data_processing/
│   └─ __init__.py
├─ models/              # Modelos entrenados y serializados
├─ reports/             # Informes, presentaciones, resultados de análisis
├─ docs/                # Documentación del proyecto
├─ .github/
├─ .gitignore
├─ README.md
├─ requirements.txt
└─ ...

```

- **Microservicios:** En un enfoque de polyrepo, cada microservicio reside en su propio repositorio. En un monorepo, pueden agruparse bajo una carpeta `services/`.

Plain Text

```

.
├─ services/             # Contiene múltiples microservicios
│   ├── user-service/    # Microservicio de usuarios (e.g., Spring Boot)
│   │   └─ src/
│   │       └─ ...
│   └─ product-service/  # Microservicio de productos (e.g., FastAPI)

```

```

├── src/
│   └── ...
└── ...
├── shared-libs/           # Librerías compartidas entre microservicios
├── docs/
├── .github/
├── .gitignore
├── README.md
└── ...

```

- **Librerías/Paquetes:** Diseñadas para ser reutilizables y publicables.

Plain Text

```

.
├── src/                   # Código fuente de la librería
│   ├── my_library/
│   │   ├── __init__.py
│   │   └── module1.py
│   └── ...
├── tests/                 # Pruebas unitarias e de integración
├── docs/                  # Documentación de la API y ejemplos
├── examples/              # Ejemplos de uso de la librería
├── .github/
├── .gitignore
├── README.md
├── LICENSE
├── pyproject.toml / setup.py
└── ...

```

Archivos "Estándar" que Todo Repositorio Debería Tener

Independientemente del tipo de proyecto, ciertos archivos son esenciales para la claridad, la legalidad y la colaboración ¹:

Archivo	Propósito
README.md	El punto de entrada principal. Explica qué es el proyecto, por qué existe y cómo empezar a usarlo.
LICENSE	Define los términos legales bajo los cuales se puede usar, modificar y distribuir el código.
CONTRIBUTING.md	Guía para los desarrolladores que desean contribuir al proyecto (estilo de código, proceso de PR, etc.).

<code>CHANGELOG.md</code>	Un registro cronológico de todos los cambios notables realizados en cada versión del proyecto.
<code>CODE_OF_CONDUCT.md</code>	Establece las normas de comportamiento esperado dentro de la comunidad del proyecto.
<code>SECURITY.md</code>	Describe las políticas de seguridad del proyecto y cómo reportar vulnerabilidades.
<code>.gitignore</code>	Especifica los archivos y directorios que Git debe ignorar (e.g., archivos compilados, dependencias locales, secretos).

Mejores Prácticas para la Carpeta `.github/`

La carpeta `.github/` es un estándar de facto para almacenar configuraciones específicas de GitHub, mejorando la automatización y la gestión de la comunidad ¹:

- **Plantillas de Issues (`ISSUE_TEMPLATE/`)**: Estandarizan la información que los usuarios proporcionan al reportar errores o solicitar características.
- **Plantillas de Pull Requests (`PULL_REQUEST_TEMPLATE.md`)**: Guían a los contribuyentes sobre qué información incluir al enviar un PR (e.g., descripción de cambios, pruebas realizadas).
- **Flujos de Trabajo (`workflows/`)**: Contiene los archivos YAML que definen las acciones de CI/CD (GitHub Actions).
- **CODEOWNERS**: Define qué individuos o equipos son responsables de revisar los cambios en partes específicas del código.

Organización a Nivel Raíz vs. Anidada

La raíz del repositorio debe mantenerse lo más limpia posible. Debe contener solo los archivos esenciales de configuración, documentación general y los directorios principales del proyecto (e.g., `src/`, `docs/`, `tests/`). Evitar el "volcado de archivos" en la raíz; agrupar archivos relacionados en subdirectorios ⁶.

Monorepo vs. Polyrepo

La elección entre un monorepo (múltiples proyectos en un solo repositorio) y un polyrepo (un repositorio por proyecto) depende de la escala y la organización del equipo ⁴.

- **Monorepo:**

- *Cuándo usarlo*: Cuando los proyectos están estrechamente acoplados, comparten muchas dependencias o cuando se desea simplificar la gestión de versiones y las refactorizaciones a gran escala.
- *Ventajas*: Facilita la compartición de código, simplifica la gestión de dependencias, permite refactorizaciones atómicas.
- *Desventajas*: Puede volverse muy grande y lento de clonar, requiere herramientas de construcción especializadas (e.g., Bazel, Nx).
- **Polyrepo**:
 - *Cuándo usarlo*: Cuando los proyectos son independientes, tienen ciclos de vida de lanzamiento separados o cuando diferentes equipos trabajan en proyectos distintos con poca superposición.
 - *Ventajas*: Límites claros de propiedad, clonación rápida, ciclos de lanzamiento independientes.
 - *Desventajas*: Dificulta la compartición de código, requiere herramientas para gestionar dependencias entre repositorios.

Ejemplos de Repositorios Bien Estructurados

- **Python**: [Django](#), [Requests](#)
- **React**: [React](#), [Next.js](#)
- **Rust**: [Cargo](#), [Ripgrep](#)
- **Java**: [Spring Boot](#)

2. Estructura de la Carpeta de Documentación

Una documentación bien organizada es crucial para la adopción y el mantenimiento a largo plazo de un proyecto. Facilita que los usuarios y colaboradores comprendan cómo funciona el sistema, cómo usarlo y cómo contribuir a él ⁹.

Cómo Organizar una Carpeta `/docs`

La carpeta `/docs` debe tener una estructura lógica que guíe al usuario desde los conceptos básicos hasta los detalles avanzados. Una estructura común podría ser:

Plain Text

```
docs/  
├─ index.md           # Página principal de la documentación, introducción  
├─ getting-started/   # Guías de inicio rápido, instalación, configuración  
└─ ── installation.md
```

```
| └─ configuration.md
└─ guides/                # Tutoriales paso a paso, cómo hacer tareas específicas
    └─ user-guide.md
    └─ advanced-topics.md
└─ api-reference/         # Documentación detallada de la API (a menudo generada)
    └─ backend-api.md
    └─ frontend-components.md
└─ architecture/         # Diagramas de arquitectura, decisiones de diseño (ADRs)
    └─ overview.md
    └─ adrs/              # Architecture Decision Records
        └─ 0001-initial-architecture.md
        └─ 0002-database-choice.md
    └─ data-flow.md
└─ contributing/         # Guías para desarrolladores (puede enlazar a CONTRIBUTING.md)
    └─ development-setup.md
    └─ testing.md
└─ faq.md                # Preguntas frecuentes
```

Jerarquía de la Documentación

La documentación debe seguir una jerarquía clara para facilitar la navegación y la comprensión:

1. **Getting Started:** Proporciona la información mínima necesaria para que un nuevo usuario o desarrollador ponga el proyecto en marcha. Incluye instrucciones de instalación, configuración básica y un ejemplo de "Hola Mundo".
2. **Guides/Tutorials:** Ofrece tutoriales paso a paso y guías prácticas para realizar tareas específicas o utilizar características clave del proyecto. Se enfoca en el "cómo hacer".
3. **API Reference:** Contiene la documentación exhaustiva y detallada de todas las interfaces de programación (clases, funciones, métodos, endpoints). A menudo se genera automáticamente a partir de los comentarios en el código fuente.
4. **Architecture:** Explica el diseño de alto nivel del sistema, sus componentes principales, las interacciones entre ellos y las decisiones clave que se tomaron durante su desarrollo. Responde al "por qué" ⁹.
5. **ADRs (Architecture Decision Records):** Son documentos concisos que capturan una única decisión arquitectónica significativa, su contexto, las opciones consideradas y las consecuencias. Son invaluable para preservar el conocimiento y entender la evolución del sistema ⁷ ⁸.

Enlaces entre Documentos — Mejores Prácticas para Referencias Cruzadas

- **Enlaces Relativos:** Utilizar rutas relativas para enlazar entre documentos dentro de la misma estructura `/docs` . Esto asegura que los enlaces sigan siendo válidos si la documentación se mueve o se publica en un sitio diferente.
- **Texto Descriptivo:** Usar texto de enlace significativo en lugar de "haz clic aquí".
- **Consistencia:** Mantener un estilo consistente para los enlaces internos.

Enfoque Docs-as-Code

Este enfoque trata la documentación con las mismas prácticas que el código fuente:

- **Control de Versiones:** La documentación se almacena en el mismo repositorio que el código y se versiona con Git.
- **Formato de Texto Plano:** Se escribe en formatos como Markdown, reStructuredText o AsciiDoc, lo que facilita la revisión y el seguimiento de cambios.
- **Revisión por Pares:** La documentación pasa por el mismo proceso de revisión por pares que el código.
- **Automatización:** Se utilizan herramientas de CI/CD para construir, probar y publicar la documentación automáticamente.

Cuándo Usar una Wiki vs. Carpeta `/docs` vs. Sitio de Documentación Externo

Característica	Wiki (e.g., GitHub Wiki)	Carpeta <code>/docs</code> (en el repo)	Sitio de Doc Externo (e.g Docusaurus)
Propósito	Notas rápidas, borradores, documentación interna, conocimiento efímero.	Documentación oficial del proyecto, versionada con el código.	Documenta profesional, característic
Control de Versiones	Limitado o menos riguroso.	Completo (Git), versionado con el código.	Completo (C con soporte versiones.
Facilidad de Edición	Muy fácil, edición en línea.	Requiere clonar el repo, editar archivos, PR.	Requiere clc editar archiv
Integración CI/CD	Generalmente no.	Sí, se puede automatizar la construcción y publicación.	Sí, es su pro principal.
Características	Búsqueda básica, historial de revisiones.	Búsqueda limitada (depende de la plataforma), enlaces internos.	Búsqueda a versionado, personaliza internacion:

Audiencia	Interna del equipo.	Interna y externa (si es un proyecto de código abierto).	Principalmente usuarios finales desarrollados
------------------	---------------------	--	---

Herramientas para Sitios de Documentación

- **MkDocs:** Un generador de sitios estáticos rápido y sencillo, ideal para documentación de proyectos. Utiliza Markdown y se configura con un único archivo YAML. Es fácil de usar y tiene una buena comunidad [12](#).
- **Docusaurus:** Desarrollado por Meta, es un generador de sitios estáticos basado en React. Ofrece características avanzadas como versionado de documentos, búsqueda integrada (Algolia DocSearch) y soporte para internacionalización. Es excelente para proyectos grandes y con necesidades de documentación complejas [12](#).
- **mdBook (para Rust):** Una herramienta similar a GitBook, escrita en Rust, que permite crear libros a partir de archivos Markdown. Es la opción preferida en el ecosistema Rust para la documentación de proyectos y crates.
- **Sphinx (para Python):** Un generador de documentación muy potente y flexible, ampliamente utilizado en la comunidad Python. Soporta reStructuredText y Markdown (a través de extensiones), y puede generar documentación de API directamente desde el código fuente.

Tabla de Contenidos y Patrones de Navegación

Una buena navegación es clave para la usabilidad de la documentación:

- **Tabla de Contenidos (TOC):** Generar automáticamente una TOC para cada página o sección larga. Esto permite a los usuarios saltar rápidamente a la información relevante.
- **Barra de Navegación Lateral:** Una barra de navegación persistente en el lado izquierdo que muestra la estructura jerárquica de la documentación.
- **Migas de Pan (Breadcrumbs):** Indican la ubicación actual del usuario dentro de la jerarquía de la documentación.
- **Búsqueda:** Una función de búsqueda eficiente es indispensable para cualquier sitio de documentación.

Documentación Versionada

Para proyectos que tienen múltiples versiones activas o que evolucionan rápidamente, el versionado de la documentación es crucial. Permite a los usuarios acceder a la documentación relevante para la versión del software que están utilizando. Herramientas

como Docusaurus y Read the Docs ofrecen soporte nativo para el versionado de la documentación.

3. División de Archivos y Organización del Código

La organización del código a nivel de archivo es fundamental para la legibilidad, el mantenimiento y la escalabilidad de un proyecto. Un código bien estructurado reduce la complejidad cognitiva y facilita la colaboración ⁶.

Cuándo Dividir un Archivo (Señales de que un archivo es demasiado grande)

Un archivo se considera "demasiado grande" cuando su tamaño o complejidad dificultan su comprensión y mantenimiento. Algunas señales de advertencia incluyen:

- **Dificultad para Encontrar Código:** Si los desarrolladores pasan mucho tiempo desplazándose o utilizando la función de búsqueda dentro de un archivo para encontrar una función o clase específica.
- **Múltiples Responsabilidades:** Si un único archivo maneja preocupaciones dispares, como la interfaz de usuario, la lógica de negocio, la persistencia de datos y la comunicación de red. Esto viola el Principio de Responsabilidad Única.
- **Demasiadas Dependencias:** Un archivo con una lista excesivamente larga de importaciones (imports) o dependencias externas es un indicio de que está intentando hacer demasiadas cosas o que tiene un acoplamiento alto.
- **Dificultad para Probar:** Si escribir pruebas unitarias para el archivo es complejo, requiere una configuración extensa o implica mockear muchas dependencias, es probable que el archivo sea demasiado grande o esté mal diseñado.
- **Líneas de Código Excesivas:** Aunque no es una métrica perfecta, un archivo que supera consistentemente las 500-1000 líneas de código es un fuerte candidato para ser dividido

⁶.

El Tamaño Ideal del Archivo — Reglas Generales

No hay una regla estricta sobre el tamaño ideal de un archivo, ya que depende del lenguaje, el paradigma de programación y la complejidad de la lógica. Sin embargo, algunas reglas de oro son:

- **Menos de 300-500 líneas:** Un buen objetivo para la mayoría de los archivos. Facilita la lectura y la comprensión de un vistazo.
- **Evitar "God Files":** Archivos de más de 1000 líneas son casi siempre un antipatrón. Suelen ser difíciles de entender, mantener y probar, y son un signo de baja cohesión ⁶.

- **Centrarse en la Cohesión:** Más importante que el número de líneas es que el archivo tenga una única responsabilidad bien definida y que todo su contenido esté estrechamente relacionado.

Cómo Dividir Archivos sin Romper Cosas

La división de archivos debe ser un proceso cuidadoso para evitar introducir errores:

1. **Identificar Responsabilidades Claras:** Analizar el archivo grande y agrupar funciones, clases o componentes que tienen una responsabilidad común o que están estrechamente relacionados lógicamente.
2. **Crear Nuevos Archivos/Módulos:** Mover cada grupo lógico identificado a un nuevo archivo o módulo con un nombre descriptivo que refleje su nueva responsabilidad.
3. **Actualizar Importaciones y Referencias:** Modificar todas las importaciones y referencias al código movido en el archivo original y en cualquier otro archivo que lo utilice. Las herramientas de refactorización del IDE pueden ser de gran ayuda aquí.
4. **Ejecutar Pruebas Exhaustivamente:** Después de cada paso de refactorización, ejecutar la suite de pruebas completa para asegurarse de que no se haya introducido ninguna regresión y que la funcionalidad existente siga funcionando correctamente.
5. **Revisión por Pares:** Solicitar una revisión por pares para validar la nueva estructura y asegurarse de que la lógica se ha dividido correctamente.

Principio de Responsabilidad Única (SRP) Aplicado a Archivos/Módulos

El SRP establece que una clase, módulo o archivo debe tener una, y solo una, razón para cambiar. Aplicado a los archivos, significa que cada archivo debe encapsular una única responsabilidad bien definida. Por ejemplo, un archivo no debería manejar tanto la autenticación de usuarios como la lógica de procesamiento de pedidos; estas deberían ser responsabilidades separadas en archivos o módulos distintos.

Cohesión y Acoplamiento a Nivel de Archivo

Estos dos principios son fundamentales para el diseño de software modular:

- **Cohesión:** Se refiere a la medida en que los elementos dentro de un módulo (o archivo) pertenecen juntos. Un archivo con **alta cohesión** significa que todas sus funciones, clases y variables están estrechamente relacionadas y trabajan juntas para lograr un propósito bien definido. Esto es deseable.
- **Acoplamiento:** Se refiere a la medida en que un módulo (o archivo) depende de otros módulos. Un **bajo acoplamiento** significa que los archivos son lo más independiente

posible entre sí, lo que facilita su modificación, prueba y reutilización. Esto también es deseable.

El objetivo es diseñar archivos con alta cohesión y bajo acoplamiento.

Límites de los Módulos — Cómo Decidir Qué Va Dónde

La decisión de dónde colocar el código es crucial. Una práctica común es agrupar el código por **característica (feature)** o dominio de negocio, en lugar de por tipo técnico (e.g., `controllers/` , `services/` , `models/`).

- **Agrupación por Característica:** Todos los componentes, servicios, modelos y pruebas relacionados con una característica específica (e.g., `user-management` , `product-catalog`) se agrupan en una única carpeta. Esto mejora la localidad del código y facilita el desarrollo de nuevas características ² .
- **Agrupación por Capa (Layer-based):** Agrupa el código por su rol técnico (e.g., `controllers/` , `services/` , `repositories/`). Puede ser útil para proyectos pequeños, pero a menudo conduce a un acoplamiento alto entre capas y dificulta la comprensión de una característica completa.

Archivos Índice/Barrel (`index.ts` , `__init__.py` , `mod.rs`)

Estos archivos sirven como puntos de entrada o "barriles" para un directorio, reexportando selectivamente los módulos internos para proporcionar una interfaz pública limpia y simplificar las importaciones. Ayudan a ocultar la complejidad interna de un directorio y a controlar lo que se expone al exterior.

- **`index.ts` (TypeScript/JavaScript):** Comúnmente utilizado para exportar componentes, funciones o tipos desde un directorio, permitiendo importaciones más limpias como `import { Button } from './components';` en lugar de `import { Button from './components/Button';` .
- **`__init__.py` (Python):** Marca un directorio como un paquete Python. Puede estar vacío o utilizarse para definir la API pública del paquete, inicializar variables o realizar importaciones de sub-módulos ⁶ .
- **`mod.rs` (Rust):** Define el módulo raíz de un directorio en Rust. En las ediciones más recientes de Rust, se puede usar `nombre_modulo.rs` en lugar de `nombre_modulo/mod.rs` para definir submódulos.

Patrones Específicos por Lenguaje

- **Python:**
 - **Paquetes y Módulos:** Utilizar directorios con archivos `__init__.py` para crear paquetes y organizar el código en módulos lógicos (`.py` files).

- **"src layout" vs. "flat layout":** Se recomienda el **"src layout"** (`src/my_package/`) donde el código fuente instalable reside en un subdirectorio `src/` . Esto ayuda a evitar problemas de importación accidental durante el desarrollo y asegura que las pruebas se ejecuten contra el código instalado, no contra el código fuente en el directorio de trabajo 3 .
- **__init__.py** : Puede estar vacío o contener importaciones para exponer una API de paquete simplificada.
- **Rust:**
 - **Crates y Módulos:** Los proyectos Rust se organizan en "crates" (binarios o librerías). Dentro de un crate, el código se organiza en módulos usando la palabra clave `mod` .
 - **mod.rs y nombre_modulo.rs** : Tradicionalmente, un archivo `mod.rs` dentro de un directorio definía el módulo para ese directorio. Las versiones más recientes de Rust permiten usar `nombre_modulo.rs` para definir un módulo llamado `nombre_modulo` y `nombre_modulo/` para sus submódulos, simplificando la estructura.
 - **Visibilidad:** Utilizar `pub` para controlar la visibilidad de los elementos dentro de los módulos.
- **TypeScript/React:**
 - **Organización de Componentes:** La organización **basada en características (feature-based)** es altamente recomendada. Cada característica (e.g., `Auth` , `Dashboard` , `UserProfile`) tiene su propia carpeta que contiene todos los componentes, hooks, servicios, tipos y estilos relacionados 2 .
 - **Componentes Reutilizables:** Los componentes genéricos y reutilizables que no pertenecen a una característica específica deben colocarse en una carpeta `shared/components` o `ui/components` .
 - **Archivos de Barril (index.ts):** Utilizar `index.ts` para exportar componentes y funciones de un directorio, simplificando las importaciones.
- **Java/Kotlin (Spring Boot):**
 - **Estructura de Paquetes:** Preferir **"Package by Feature"** (paquetes por característica de negocio, e.g., `com.mycompany.myapp.user` , `com.mycompany.myapp.order`) sobre "Package by Layer" (paquetes por capa técnica, e.g., `com.mycompany.myapp.controller` , `com.mycompany.myapp.service`). La agrupación por característica mejora la cohesión y reduce el acoplamiento.
 - **Una Clase por Archivo:** Seguir la convención de Java/Kotlin de tener una única clase pública por archivo, con el nombre del archivo coincidiendo con el nombre de la clase.

- **Configuración:** Mantener los archivos de configuración de Spring Boot (e.g., `application.properties` , `application.yml`) en `src/main/resources` .

4. Convenciones de Nomenclatura para Archivos y Carpetas

La consistencia en la nomenclatura es un pilar fundamental para la legibilidad y el mantenimiento de cualquier proyecto de software. Un sistema de nombres bien definido reduce la ambigüedad y facilita la navegación y comprensión del código ⁵ .

Patrones de Nomenclatura de Archivos

La elección del patrón de nomenclatura a menudo depende del lenguaje de programación y del contexto del archivo:

- **kebab-case (guiones):** `mi-archivo-de-ejemplo.md` , `user-profile.component.ts`
 - **Cuándo usarlo:** Común para nombres de archivos en proyectos web (HTML, CSS, Markdown), nombres de componentes en frameworks frontend (React, Angular) y a menudo en TypeScript/JavaScript. También es adecuado para archivos de configuración.
- **snake_case (guiones bajos):** `mi_archivo_de_ejemplo.py` , `user_auth_service.rs`
 - **Cuándo usarlo:** El estándar en Python y Rust para nombres de archivos y módulos. También se utiliza para variables y funciones en estos lenguajes.
- **PascalCase (CamelCase con mayúscula inicial):** `UserProfile.java` , `UserService.kt` , `ButtonComponent.tsx`
 - **Cuándo usarlo:** Común en Java/Kotlin para nombres de archivos que contienen clases o interfaces. En TypeScript/React, se usa para nombres de componentes.

Convenciones de Nomenclatura de Carpetas

Las carpetas generalmente deben usar `kebab-case` o `snake_case` .

- **kebab-case :** `user-management/` , `data-processing/`
 - **Cuándo usarlo:** Preferido en muchos proyectos web y JavaScript/TypeScript. Es compatible con la mayoría de los sistemas de archivos y evita problemas de distinción entre mayúsculas y minúsculas.
- **snake_case :** `user_management/` , `data_processing/`
 - **Cuándo usarlo:** Común en proyectos Python y Rust.

Evitar Nombres Ambiguos

Utilizar nombres descriptivos que indiquen claramente el propósito y el contenido del archivo o carpeta. Evitar nombres genéricos o abreviaturas crípticas.

- **Malo:** `utils.py` , `helpers.js` , `data.json`
- **Bueno:** `string_utils.py` , `date_helpers.ts` , `user_config.json`

Consistencia en la Nomenclatura en Todo el Proyecto

La clave es la consistencia. Una vez que se elige una convención para un tipo de archivo o carpeta, se debe aplicar rigurosamente en todo el proyecto. Esto se puede hacer mediante:

- **Guías de Estilo:** Documentar las convenciones de nomenclatura en la guía de estilo del proyecto.
- **Revisiones de Código:** Asegurarse de que los revisores hagan cumplir las convenciones.
- **Linters y Herramientas de Formateo:** Configurar herramientas para detectar y corregir inconsistencias automáticamente.

Convenciones Específicas por Lenguaje

- **Python:** `snake_case` para nombres de módulos y paquetes. Los nombres de clases usan `PascalCase` .
- **Rust:** `snake_case` para nombres de módulos y archivos. Los nombres de tipos (structs, enums) usan `PascalCase` .
- **TypeScript/React:** `kebab-case` para nombres de archivos de componentes y módulos. `PascalCase` para nombres de componentes y tipos. `camelCase` para variables y funciones.
- **Java/Kotlin:** `PascalCase` para nombres de clases y archivos. `camelCase` para variables y métodos.

5. Mejores Prácticas para README (Avanzado)

El archivo `README.md` es la primera impresión que un usuario o contribuyente potencial tiene de un proyecto. Un README bien elaborado es crucial para la adopción, la comprensión y el fomento de la comunidad ¹ ⁵ .

La Estructura Perfecta del README

Un `README.md` completo y efectivo debe incluir los siguientes elementos, idealmente en este orden:

1. **Título y Descripción Breve:** Un título claro del proyecto y una descripción concisa que explique qué es el proyecto, su propósito principal y el problema que resuelve.

2. **Insignias (Badges):** Pequeñas imágenes que muestran el estado del proyecto (e.g., estado de la compilación, cobertura de código, versión actual, licencia, número de descargas). Servicios como Shields.io son excelentes para esto.
3. **Tabla de Contenidos (TOC):** Para READMEs largos, una TOC facilita la navegación y permite a los lectores saltar directamente a las secciones de interés.
4. **Capturas de Pantalla y GIFs:** Para proyectos con interfaz de usuario o herramientas CLI, incluir capturas de pantalla o GIFs animados que demuestren el proyecto en acción. Una imagen vale más que mil palabras.
5. **Instalación/Inicio Rápido:** Instrucciones claras y concisas sobre cómo instalar el proyecto y ponerlo en marcha rápidamente. Debe ser lo más sencillo posible.
6. **Uso:** Ejemplos básicos de cómo utilizar las funcionalidades clave del proyecto. Incluir fragmentos de código si es relevante.
7. **Características Principales:** Una lista o descripción de las funcionalidades más importantes que ofrece el proyecto.
8. **Contribución:** Una sección que invite a los usuarios a contribuir, con un enlace claro al archivo `CONTRIBUTING.md` para obtener instrucciones detalladas.
9. **Licencia:** Información sobre la licencia del proyecto, a menudo con un enlace al archivo `LICENSE` completo.
10. **Contacto/Soporte:** Cómo contactar a los mantenedores o dónde obtener soporte.
11. **Agradecimientos:** Reconocimiento a contribuyentes, proyectos de código abierto utilizados o patrocinadores.

Insignias y Escudos (Badges and Shields)

Las insignias proporcionan información de estado de un vistazo. Ejemplos comunes incluyen:

- `![Build Status](https://img.shields.io/github/actions/workflow/status/owner/repo/ci.yml?branch=main)`
- `![Coverage](https://img.shields.io/codecov/c/github/owner/repo)`
- `![Version](https://img.shields.io/github/v/release/owner/repo)`
- `![License](https://img.shields.io/github/license/owner/repo)`

Capturas de Pantalla y GIFs

Son especialmente útiles para proyectos visuales. Un GIF animado que muestra una característica clave puede ser mucho más efectivo que una larga descripción textual.

Inicio Rápido vs. Documentación Completa

El `README` debe servir como un punto de entrada rápido. Debe proporcionar suficiente información para que un usuario entienda el proyecto y comience a usarlo. La documentación completa, con detalles exhaustivos, referencia de API y guías avanzadas, debe residir en la carpeta `/docs` o en un sitio de documentación dedicado.

README para Diferentes Audiencias

Considerar que el README puede ser leído por diferentes audiencias:

- **Usuarios:** Buscan cómo instalar, usar y qué problemas resuelve el proyecto.
- **Contribuyentes:** Necesitan saber cómo configurar el entorno de desarrollo, cómo ejecutar pruebas y cómo enviar contribuciones.
- **Mantenedores:** Pueden necesitar información sobre el proceso de lanzamiento, la gestión de la comunidad, etc.

Aunque el README principal debe ser general, se pueden enlazar secciones específicas o archivos dedicados (e.g., `CONTRIBUTING.md`) para cada audiencia.

Plantillas y Generadores de README

Utilizar plantillas de README (disponibles en GitHub o en herramientas como `readme-md-generator`) puede ayudar a asegurar que todos los elementos importantes estén presentes y que la estructura sea consistente.

READMEs Multilingües

Para proyectos con una audiencia global, ofrecer versiones del README en varios idiomas (e.g., `README.es.md`, `README.zh.md`) puede mejorar la accesibilidad. GitHub permite configurar un `README.md` por defecto y luego enlazar a las versiones traducidas.

6. Changelog y Versionado

Un registro de cambios (changelog) claro y un esquema de versionado consistente son cruciales para comunicar la evolución de un proyecto a sus usuarios y colaboradores ¹.

Formato "Keep a Changelog"

El estándar [Keep a Changelog](#) es una convención popular para estructurar el archivo `CHANGELOG.md`. Propone organizar los cambios por versión y categorizarlos bajo los siguientes encabezados:

- **Added** : Para nuevas funcionalidades.

- **Changed** : Para cambios en la funcionalidad existente.
- **Deprecated** : Para características que pronto serán eliminadas.
- **Removed** : Para características eliminadas en esta versión.
- **Fixed** : Para correcciones de errores.
- **Security** : Para vulnerabilidades de seguridad.

Cada versión debe tener una fecha y un enlace a las diferencias (diff) con la versión anterior.

Versionado Semántico (SemVer)

[Semantic Versioning \(SemVer\)](#) es un esquema de versionado ampliamente adoptado que utiliza un formato `MAJOR.MINOR.PATCH` :

- **MAJOR (X.y.z)**: Incrementa cuando se realizan cambios incompatibles en la API.
- **MINOR (x.Y.z)**: Incrementa cuando se añade funcionalidad de manera compatible con versiones anteriores.
- **PATCH (x.y.Z)**: Incrementa cuando se realizan correcciones de errores compatibles con versiones anteriores.

El uso de SemVer permite a los usuarios comprender el impacto de una nueva versión y gestionar sus dependencias de manera más efectiva.

Generación Automatizada de Changelog

Herramientas como `semantic-release` (para JavaScript/TypeScript) o `standard-version` pueden automatizar la generación del `CHANGELOG.md` y el incremento de la versión SemVer basándose en los mensajes de commit. Esto reduce el esfuerzo manual y asegura la consistencia.

Conventional Commits

La especificación [Conventional Commits](#) es una convención ligera sobre los mensajes de commit que proporciona un conjunto de reglas para crear un historial de commits explícito y legible tanto por humanos como por máquinas ¹¹. Un mensaje de commit convencional tiene el formato:

Plain Text

```
<tipo>[ámbito opcional]: <descripción>
```

```
[cuerpo opcional]
```

[pie de página opcional(es)]

- **tipo** : Obligatorio, como `feat` (nueva característica), `fix` (corrección de error), `docs` (cambios en la documentación), `chore` (cambios de mantenimiento), etc.
- **ámbito** : Opcional, indica la parte del código afectada (e.g., `feat(parser):`).
- **descripción** : Breve resumen del cambio.
- **cuerpo** : Descripción más detallada.
- **pie de página** : Para referencias a issues, o para indicar `BREAKING CHANGE` .

Los Conventional Commits son la base para la generación automatizada de changelogs y la determinación de la versión SemVer.

Mejores Prácticas para Notas de Lanzamiento (Release Notes)

Las notas de lanzamiento deben ser un resumen conciso y legible de los cambios clave en una versión. Deben destacar:

- Nuevas características y mejoras.
- Correcciones de errores importantes.
- Cualquier cambio que rompa la compatibilidad (breaking changes) y cómo migrar.
- Agradecimientos a los contribuyentes.

7. Archivos de Contribución y Comunidad

Fomentar una comunidad activa y saludable es vital para el éxito a largo plazo de un proyecto de código abierto. Los archivos de contribución y comunidad establecen las expectativas y guían a los participantes ¹ .

Mejores Prácticas para `CONTRIBUTING.md`

Este archivo es una guía esencial para cualquier persona que desee contribuir al proyecto. Debe ser claro, conciso y fácil de seguir:

- **Cómo Empezar**: Instrucciones sobre cómo clonar el repositorio, configurar el entorno de desarrollo local y ejecutar el proyecto.
- **Estándares de Código**: Directrices sobre el estilo de código, convenciones de nomenclatura y cualquier otra regla de formato. Enlazar a configuraciones de linters o formateadores.
- **Proceso de Pull Request (PR)**: Explicar el flujo de trabajo para enviar un PR, incluyendo cómo crear una rama, realizar commits, abrir el PR y responder a las revisiones.

- **Ejecución de Pruebas:** Cómo ejecutar las pruebas unitarias, de integración y de extremo a extremo.
- **Reporte de Errores:** Enlazar a la plantilla de issues para reportar errores.
- **Solicitud de Características:** Enlazar a la plantilla de issues para solicitar nuevas características.

CODE_OF_CONDUCT.md

Un Código de Conducta establece las normas de comportamiento esperado para todos los participantes del proyecto. Ayuda a crear un entorno acogedor e inclusivo. El [Contributor Covenant](#) es un código de conducta ampliamente adoptado y recomendado.

SECURITY.md

Este archivo es crucial para la seguridad del proyecto. Debe proporcionar instrucciones claras sobre cómo los usuarios y los investigadores de seguridad pueden reportar vulnerabilidades de forma privada y responsable, en lugar de publicarlas en issues públicos. También puede incluir información sobre las versiones del proyecto que reciben soporte de seguridad.

SUPPORT.md

El archivo `SUPPORT.md` informa a los usuarios dónde pueden obtener ayuda con el proyecto. Esto ayuda a dirigir las preguntas de soporte fuera de los issues de GitHub, que están destinados a errores y solicitudes de características. Puede incluir enlaces a:

- Foros de la comunidad (e.g., Stack Overflow, Discord, Reddit).
- Documentación oficial.
- Canales de chat.

Plantillas de Issues y PRs que Realmente Ayudan

Las plantillas personalizadas para issues y pull requests, ubicadas en la carpeta `.github/ISSUE_TEMPLATE/` y `.github/PULL_REQUEST_TEMPLATE.md` respectivamente, son herramientas poderosas para estandarizar la información y reducir el tiempo de ida y vuelta. Deben solicitar información específica y relevante para cada tipo de issue o PR.

- **Plantilla de Issue de Bug:** Solicitar pasos para reproducir, comportamiento esperado, comportamiento actual, versión del software, entorno, capturas de pantalla.
- **Plantilla de Issue de Característica:** Solicitar una descripción de la característica, casos de uso, beneficios esperados, posibles soluciones.

- **Plantilla de Pull Request:** Solicitar una descripción de los cambios, justificación, pruebas realizadas, impacto en la documentación, lista de verificación de PR.

8. Archivos de CI/CD y Automatización

La Integración Continua (CI) y la Entrega Continua (CD) son prácticas esenciales para garantizar la calidad del código, automatizar las pruebas y acelerar el ciclo de lanzamiento. GitHub Actions es una herramienta popular para implementar CI/CD directamente en el repositorio ¹⁰.

Organización de Flujos de Trabajo de GitHub Actions

Los flujos de trabajo de GitHub Actions se definen en archivos YAML dentro de la carpeta `.github/workflows/`. Una buena organización es clave a medida que el número de flujos de trabajo crece:

- **Agrupar por Propósito:** Crear archivos YAML separados para diferentes propósitos (e.g., `ci.yml` para pruebas y linting, `deploy-staging.yml` para despliegue en staging, `release.yml` para lanzamientos).
- **Nombres Descriptivos:** Utilizar nombres de archivo claros y descriptivos (e.g., `build-and-test.yml`, `deploy-to-production.yml`).
- **Flujos de Trabajo Reutilizables:** Para evitar la duplicación, definir flujos de trabajo reutilizables que pueden ser llamados desde otros flujos de trabajo. Esto es especialmente útil en monorepos o para tareas comunes en una organización ¹⁰.

Estructura de `.github/workflows/`

Plain Text

```
.
├── workflows/
│   ├── ci.yml                # Pruebas, linting y construcción en cada push
│   ├── deploy-staging.yml    # Despliegue automático a staging
│   ├── deploy-production.yml # Despliegue manual o con aprobación a producción
│   ├── release.yml           # Creación de releases y publicación de paquete
│   ├── security-scan.yml     # Escaneos de seguridad programados
│   ├── dependency-update.yml # Actualización de dependencias (e.g., Dependabot)
│   └── reusable-build.yml     # Flujo de trabajo reutilizable para la construcción
```

Hooks de Pre-commit

Los hooks de pre-commit son scripts que se ejecutan automáticamente antes de que se realice un commit. Son una excelente manera de hacer cumplir los estándares de código y

detectar errores tempranamente, antes de que el código llegue al repositorio remoto 13.

- **Herramientas:** `pre-commit` (para Python, pero soporta múltiples lenguajes) es una herramienta popular para gestionar estos hooks.
- **Uso:** Configurar hooks para formatear el código (e.g., Black, Prettier), ejecutar linters (e.g., ESLint, Flake8), verificar tipos (e.g., MyPy) o realizar pequeñas comprobaciones de seguridad.

Configuraciones de Linting y Formateo

Las configuraciones para linters y formateadores deben estar en la raíz del proyecto para asegurar que todos los desarrolladores sigan los mismos estándares de código. Ejemplos:

- **Python:** `.flake8` , `pyproject.toml` (para Black, Ruff).
- **JavaScript/TypeScript:** `.eslintrc.js` , `.prettierrc.json` , `tsconfig.json` .
- **Java/Kotlin:** `.editorconfig` , configuraciones de Checkstyle o Ktlint.
- **Rust:** `rustfmt.toml` .

Configuración de Dependabot/Renovate

Estas herramientas automatizan la actualización de dependencias, lo que es crucial para la seguridad y para mantenerse al día con las últimas características. Se configuran en archivos como `.github/dependabot.yml` .

9. Gestión de Archivos de Configuración

La gestión adecuada de los archivos de configuración es vital para la seguridad, la portabilidad y la facilidad de despliegue de una aplicación.

Dónde Poner los Archivos de Configuración

- **Raíz del Repositorio:** Archivos de configuración generales del proyecto que son comunes a todos los entornos y necesarios para la construcción o el desarrollo local (e.g., `docker-compose.yml` , `Makefile` , `package.json` , `pyproject.toml` , `Cargo.toml`).
- **Carpetas Específicas:** Configuraciones específicas de un módulo o servicio pueden residir dentro de su respectiva carpeta (e.g., `backend/src/main/resources/application.yml` para Spring Boot).

Configuraciones Específicas del Entorno

Las configuraciones que varían entre entornos (desarrollo, pruebas, staging, producción) deben gestionarse cuidadosamente:

- **Variables de Entorno:** La forma más segura y flexible de manejar configuraciones específicas del entorno. Las aplicaciones deben leer la configuración de las variables de entorno (e.g., `DATABASE_URL` , `API_KEY`).
- **Archivos `.env`** : Para el desarrollo local, se pueden usar archivos `.env` (que deben ser ignorados por Git) para cargar variables de entorno. Herramientas como `python-dotenv` o `dotenv` (Node.js) facilitan esto.
- **Perfiles de Configuración:** Frameworks como Spring Boot permiten definir perfiles de configuración (e.g., `application-dev.yml` , `application-prod.yml`) que se activan según el entorno.

Gestión de Secretos (Qué NO Confirmar)

NUNCA se deben confirmar secretos (contraseñas, claves API, tokens, certificados, claves privadas) directamente en el repositorio de Git, incluso si es privado. Esto es una vulnerabilidad de seguridad crítica.

- **Uso de Gestores de Secretos:** Utilizar soluciones dedicadas como GitHub Secrets, HashiCorp Vault, AWS Secrets Manager, Azure Key Vault o Google Secret Manager para almacenar y acceder a los secretos de forma segura en entornos de CI/CD y producción.
- **Archivos `.env` Locales:** Para el desarrollo, usar archivos `.env` locales que contengan los secretos, pero asegurarse de que `.env` esté en el `.gitignore` .

Mejores Prácticas para `.gitignore`

Un archivo `.gitignore` bien mantenido es esencial para evitar que archivos innecesarios o sensibles se confirmen en el repositorio:

- **Plantillas Estándar:** Utilizar plantillas `.gitignore` generadas para el lenguaje y el IDE (e.g., de [github.com](https://github.com/github/gitignore) o las plantillas de GitHub).
- **Ignorar Archivos Generados:** Archivos compilados (e.g., `.class` , `.o`), directorios de dependencias (e.g., `node_modules/` , `target/` , `venv/`), archivos de registro (`*.log`).
- **Ignorar Archivos de Configuración Local:** Archivos `.env` , configuraciones de IDE personales (e.g., `.vscode/` si contiene ajustes personales, aunque a veces se comparten configuraciones de equipo).
- **Ignorar Archivos Temporales:** Archivos creados por el sistema operativo o herramientas temporales.

Configuraciones del Editor (`.editorconfig` , `.vscode/`)

- **`.editorconfig`** : Un archivo `.editorconfig` en la raíz del proyecto ayuda a mantener estilos de codificación consistentes (indentación, finales de línea, codificación de caracteres)

entre diferentes editores e IDEs para todos los desarrolladores del equipo.

- **.vscode/** : Esta carpeta puede contener configuraciones específicas de VS Code. Si se utiliza para configuraciones de equipo (e.g., recomendaciones de extensiones, configuraciones de depuración compartidas), puede incluirse en el control de versiones. Sin embargo, los archivos de configuración de usuario (`settings.json` , `keybindings.json`) deben ignorarse.

10. Patrones para Tipos de Proyectos Específicos

La estructura del repositorio debe adaptarse a la naturaleza y los requisitos del proyecto. A continuación, se presentan patrones comunes para tipos de proyectos específicos.

Monorepo Fullstack (Frontend + Backend + Shared)

Un monorepo fullstack agrupa el código del frontend, backend y cualquier código compartido en un único repositorio. Esto es común en empresas con equipos grandes y múltiples aplicaciones interconectadas.

Plain Text

```
.
├── apps/                                # Contiene las aplicaciones principales
│   ├── frontend/                       # Aplicación frontend (e.g., React)
│   │   ├── src/
│   │   ├── package.json
│   │   └── ...
│   └── backend/                        # Aplicación backend (e.g., Spring Boot, FastAPI)
│       ├── src/
│       ├── pom.xml / requirements.txt
│       └── ...
├── packages/                           # Contiene librerías y componentes compartidos
│   ├── ui-components/                 # Componentes de UI reutilizables
│   │   ├── src/
│   │   └── package.json
│   ├── shared-types/                 # Definiciones de tipos compartidas (TypeScript)
│   │   ├── src/
│   │   └── package.json
│   └── ...
├── tools/                             # Scripts y herramientas de desarrollo
├── docs/                              # Documentación del monorepo
├── .github/
├── .gitignore
└── README.md
```

```
|— package.json          # Configuración del monorepo (e.g., Lerna, Nx)
|— ...
```

Herramientas como [Nx](#), [Turborepo](#) o [Lerna](#) son esenciales para gestionar las dependencias, la construcción y las pruebas en un monorepo de manera eficiente.

Estructura de Proyecto ML/AI

Los proyectos de Machine Learning e Inteligencia Artificial tienen necesidades específicas para la gestión de datos, modelos y experimentos. El estándar **Cookiecutter Data Science** es una plantilla muy recomendada ¹⁴.

Plain Text

```
.
|— data/                  # Datos del proyecto
|   |— 01_raw/            # Datos originales, inmutables
|   |— 02_intermediate/  # Datos transformados, intermedios
|   |— 03_primary/       # Datos limpios y finales para modelado
|   |— 04_models/        # Datos de entrada/salida de modelos
|— notebooks/            # Jupyter notebooks para exploración y experimentación
|   |— exploratory/
|   |— modeling/
|— src/                  # Código fuente de la aplicación/librería
|   |— data/             # Scripts para la ingesta y procesamiento de datos
|   |— features/          # Scripts para la ingeniería de características
|   |— models/            # Scripts para la construcción y entrenamiento de modelos
|   |— visualization/    # Scripts para visualización
|— models/               # Modelos entrenados y serializados
|— reports/              # Informes, presentaciones, resultados de análisis
|   |— figures/
|   |— final/
|— docs/                 # Documentación del proyecto ML
|— tests/                # Pruebas unitarias e de integración
|— .github/
|— .gitignore
|— README.md
|— requirements.txt      # Dependencias del proyecto
|— ...
```

Estructura de Repositorio de Microservicios

Si se opta por un enfoque de polyrepo, cada microservicio reside en su propio repositorio. Si se utiliza un monorepo, los microservicios se organizan en subdirectorios:

Plain Text

```

.
├── services/                # Contiene múltiples microservicios
│   ├── user-service/       # Microservicio de usuarios (e.g., Spring Boot, FastAPI)
│   │   ├── src/
│   │   ├── Dockerfile
│   │   ├── application.yml
│   │   └── ...
│   ├── product-service/    # Microservicio de productos
│   │   ├── src/
│   │   ├── Dockerfile
│   │   └── ...
│   └── notification-service/ # Microservicio de notificaciones
│       ├── src/
│       ├── Dockerfile
│       └── ...
├── shared-libs/            # Librerías compartidas entre microservicios (si aplica)
├── docs/
├── .github/
├── .gitignore
├── README.md
└── docker-compose.yml      # Para desarrollo local de múltiples servicios

```

Cada microservicio debe ser independiente en su construcción, prueba y despliegue.

Estructura de Crate/Workspace en Rust

En Rust, los proyectos se organizan en "crates". Un "workspace" de Cargo permite gestionar múltiples crates relacionados dentro de un solo repositorio.

Plain Text

```

.
├── Cargo.toml              # Configuración del workspace
├── crates/                 # Contiene los crates individuales
│   ├── my-app/             # Crate binario (aplicación ejecutable)
│   │   ├── src/
│   │   │   └── main.rs
│   │   └── Cargo.toml
│   ├── my-library/         # Crate de librería
│   │   ├── src/
│   │   │   └── lib.rs
│   │   └── Cargo.toml
│   └── my-utils/           # Otro crate de librería
│       ├── src/
│       │   └── lib.rs
│       └── Cargo.toml
└── docs/

```



```
|— .github/
|— .gitignore
|— README.md
|— pyproject.toml / setup.py
|— ...
```

11. Antipatrones a Evitar

Identificar y evitar los antipatrones comunes en la estructura y organización del código es tan importante como seguir las mejores prácticas. Estos errores pueden degradar la calidad del proyecto y dificultar su mantenimiento a largo plazo.

- **Archivos "Dios" (God files):** Archivos con miles de líneas de código (e.g., 1000+ líneas) que asumen múltiples responsabilidades. Indican baja cohesión y alta complejidad. Dificultan la lectura, el mantenimiento y las pruebas. Dividir estos archivos en unidades más pequeñas y con una única responsabilidad ⁶.
- **Estructura Plana con Todo en la Raíz:** Dificulta la navegación y la comprensión de la arquitectura del proyecto, especialmente a medida que crece. Agrupar archivos relacionados en subdirectorios lógicos.
- **Nomenclatura Inconsistente:** Causa confusión y ralentiza el desarrollo. Establecer y seguir convenciones de nomenclatura estrictas para archivos, carpetas, variables y funciones.
- **Código Muerto y Archivos Sin Usar:** Mantener el repositorio limpio y relevante eliminando el código y los archivos que ya no son necesarios. Esto reduce la complejidad y el tamaño del repositorio.
- **Documentación Nunca Actualizada:** La documentación desactualizada es peor que la ausencia de documentación, ya que puede llevar a errores y frustración. Integrar la actualización de la documentación en el flujo de trabajo de desarrollo (docs-as-code) y asegurarse de que se revise junto con los cambios de código.
- **Anidamiento Excesivo (Jerarquías de Carpetas Demasiado Profundas):** Dificulta la navegación y puede hacer que las rutas de los archivos sean excesivamente largas. Buscar un equilibrio entre la organización y la profundidad, manteniendo las jerarquías lo más planas posible sin sacrificar la claridad.
- **Mezclar Preocupaciones en la Misma Carpeta:** Por ejemplo, tener archivos de UI, lógica de negocio y acceso a datos en el mismo directorio. Separar las preocupaciones en carpetas lógicas para mejorar la cohesión y reducir el acoplamiento.

12. Herramientas y Automatización

La automatización y el uso de herramientas adecuadas pueden simplificar la aplicación de las mejores prácticas y mantener la consistencia en el proyecto.

- **Comando `tree`** : Una utilidad de línea de comandos (disponible en sistemas Unix-like) que permite visualizar la estructura de directorios de forma recursiva. Es muy útil para auditar la estructura de un repositorio y para comunicarla visualmente a otros miembros del equipo.

Bash

```
tree -L 2 # Muestra la estructura hasta 2 niveles de profundidad
```

- **Plantillas de Repositorio (GitHub Template Repos)**: GitHub permite marcar un repositorio como plantilla, lo que facilita la creación de nuevos repositorios con una estructura inicial predefinida, archivos estándar y configuraciones básicas. Esto asegura una consistencia desde el inicio del proyecto.
- **Cookiecutter / `cargo-generate` / `create-react-app`** : Estas son herramientas de línea de comandos que generan proyectos a partir de plantillas predefinidas para lenguajes y frameworks específicos:
 - **Cookiecutter (Python)**: Genera proyectos Python a partir de plantillas, como la popular [Cookiecutter Data Science](#) ¹⁴.
 - **`cargo-generate` (Rust)**: Una herramienta similar para proyectos Rust.
 - **`create-react-app` (React)**: Una herramienta oficial para configurar un proyecto React moderno sin configuración de construcción.
- **Linters que Imponen Estructura**: Algunos linters pueden configurarse para hacer cumplir reglas relacionadas con la estructura del proyecto y las importaciones. Por ejemplo, `eslint-plugin-import` para JavaScript/TypeScript puede ayudar a mantener un orden consistente en las importaciones y a prevenir importaciones circulares.
- **Generadores de Documentación**: Herramientas como MkDocs, Docusaurus, mdBook y Sphinx automatizan la creación de sitios de documentación a partir de archivos Markdown o comentarios en el código fuente. Esto facilita la publicación de documentación profesional y actualizada.

Referencias

[1] GitHub Repository Structure Best Practices - Code Factory Berlin

[2] 3 Folder Structures in React I've Used — And Why Feature-Based Is My Favorite - Asrul Kadir

[3] src layout vs flat layout - Python Packaging User Guide

- [4] Monorepo vs. Polyrepo: How to Choose Between Them - Buildkite
- [5] GitHub Repository Best Practices - Marcel.L
- [6] Structuring Your Project - The Hitchhiker's Guide to Python
- [7] Architecture decision record (ADR) - GitHub
- [8] A practical guide to Architecture Decision Records (ADRs) - Dr Milan Milanovic
- [9] Architecture Docs: A Guide to Clear, Effective System Documentation - ClickHelp
- [10] Github Actions Workflows: GitHub Actions Patterns & Best Practices - Thesius Code
- [11] Conventional Commits
- [12] MkDocs vs Docusaurus for technical documentation - Damavis Blog
- [13] pre-commit
- [14] Cookiecutter Data Science - DrivenData